CHAPTER 08

# Design A Unique ID Generator In Distributed Systems

Generate globally unique, roughly time-sortable 64-bit IDs without a central auto-increment.

**Jump to:** Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

## 1. Problem statement

A relational DB's `auto_increment` does not work when data is sharded across many servers: one DB cannot hand out IDs at scale, and per-DB counters would collide. Design a service that generates **unique, sortable IDs** at very high rate across many machines and data centers.

**Why interviewers ask it:** A short, well-bounded problem with several known solutions; shows you can reason about clocks, bit layouts and coordination.

## 2. Clarifying questions to ask (and sensible assumptions)

| Question | Assumed answer |
|---|---|
| Characteristics of IDs? | Unique and sortable by time (roughly). |
| Numeric only? | Yes. |
| ID length? | 64 bits. |
| Scale? | 10,000 IDs per second. |
| Increment by 1? | No - just increase with time. |

## 3. Requirements

### Functional

- Generate unique IDs.
- IDs are numerical, fit in 64 bits, and are ordered by creation time.

### Non-functional

- Very high throughput (10 K/s now, headroom to millions/s).
- Low latency, ideally generated locally with no network call.
- Highly available, no single point of failure, no coordination on the hot path.

## 4. Back-of-the-envelope estimation

- 10,000 IDs/s x 86,400 = about 864 M IDs/day = about 315 B IDs/year.
- 41 timestamp bits = 2^41 ms = about 69 years of IDs; 12 sequence bits = 4,096 IDs per ms per machine; 10 machine bits = 1,024 machines -> about 4 M IDs/ms cluster-wide.

## 5. High-level design

### Snowflake 64-bit layout

Sort by ID == sort by time (timestamp is most significant). Sequence resets every millisecond.

## 6. Deep dive

### Option 1: Multi-master replication

- Use the DB `auto_increment` but step by k (number of DB servers): server 1 gives 1,3,5...; server 2 gives 2,4,6.
- **Pros:** simple, uses existing DBs. **Cons:** hard to scale across data centers; IDs do not go up with time across servers; breaks when servers are added/removed.

### Option 2: UUID (128-bit)

- Each server generates UUIDs independently; collision probability is negligible (about 1 in a billion after generating 1 B UUIDs per second for ~100 years).
- **Pros:** no coordination, trivially scalable. **Cons:** 128 bits (not 64), not time-sortable, non-numeric, poor B-tree index locality. (UUIDv7 / ULID fix sortability.)

### Option 3: Ticket server (Flickr)

- A centralised DB with `auto_increment` that all web servers call. **Pros:** numeric, easy, good for small/medium apps. **Cons:** single point of failure; multiple ticket servers bring sync problems.

### Option 4: Twitter Snowflake (recommended)

- Divide 64 bits into sections: **sign (1) + timestamp (41) + datacenter ID (5) + machine ID (5) + sequence (12)**.
- Timestamp: milliseconds since a custom epoch (e.g. Nov 04, 2010 for Twitter) -> sortable, ~69 years of range.
- Datacenter/machine ID: fixed at startup (assigned via config or ZooKeeper) -> up to 32 DCs x 32 machines.
- Sequence: increments for IDs generated in the same ms on the same machine, resets to 0 each ms; if it overflows, wait for the next ms.

```
id = ((now_ms - EPOCH) << 22) | (dc << 17) | (machine << 12) | sequence
```

### Clock synchronisation

- Snowflake assumes all machines agree on time. Real clocks drift and can jump backwards (NTP corrections).
- Mitigations: run NTP; if the clock moves backwards, **refuse to generate and wait** (or throw) until it catches up; keep the last timestamp in memory/disk.
- Multi-core / multi-socket machines may differ slightly - NTP is the accepted answer.

### Tuning the layout

- Fewer sequence bits and more timestamp bits for low-concurrency, long-lived apps; more sequence bits for high concurrency.
- Section lengths are tunable; e.g. drop DC bits if you have one DC and give them to machine bits.

### High availability

- Run the ID generator as a library embedded in each service, or a small stateless service behind a load balancer; no shared state is needed.

## 7. Trade-offs and alternatives

| Decision | Option A / Option B | How to choose |
|---|---|---|
| Approach | Auto-increment step / UUID / Ticket server / Snowflake | Snowflake meets all stated needs: 64-bit, sortable, decentralised, high throughput. |
| Sortability | Strict global order / Roughly time-ordered | Roughly time-ordered is enough for feeds and pagination; strict order requires a single sequencer (does not scale). |
| ID source | Embedded library / Dedicated service | Library = no network hop; service = central control of machine IDs. |

## 8. Scaling, reliability, failure modes

- No hot path coordination: each machine produces IDs independently from (time, machine ID, sequence).
- Machine IDs must be unique at all times; assign via ZooKeeper/etcd lease or config; reclaim after crashes carefully.
- Monitor clock skew alerts and sequence exhaustion.

## 9. Likely follow-up questions

- What if the clock goes backwards?
- How many IDs per ms per machine and what if you exceed it?
- How do you assign unique machine IDs automatically?
- How would you shorten IDs for a URL shortener? (Base62 encoding)

## 10. Common mistakes

- Forgetting clock skew and backwards movement.
- Using a timestamp resolution that causes collisions without a sequence.
- Duplicate machine IDs after a container restart.

## 11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

| Phase | Time | What to say |
|---|---|---|
| 1. Understand + scope | 4 min | "Auto-increment fails once the database is sharded. So I need IDs that are unique, numeric, 64-bit and sortable by time, at about 10,000 per second, generated without a network hop. Do we need strictly increasing IDs, or just roughly time-ordered? I'll assume roughly time-ordered." |
| 2. High-level design | 8 min | "I compared four options: multi-master auto-increment with a step, UUID, a Flickr-style ticket server, and Twitter Snowflake. UUIDs are 128 bits and not sortable, the ticket server is a single point of failure, so I'll choose Snowflake and embed it as a library in each service." |
| 3. Deep dive | 25 min | "The 64 bits are one sign bit, 41 bits of millisecond timestamp, 5 bits of data center, 5 bits of machine ID and a 12-bit sequence. That gives 69 years, 1,024 machines and 4,096 IDs per millisecond per machine. The risks are clock drift and clock moving backwards: I run NTP, remember the last timestamp, and refuse to issue IDs until the clock catches up. Machine IDs are assigned via ZooKeeper or config." |
| 4. Wrap up | 4 min | "This meets uniqueness, sortability and throughput without coordination. I'd monitor clock skew and sequence overflow, and the bit layout can be retuned: fewer sequence bits for low-traffic systems, more machine bits for bigger fleets." |

## 12. 60-second recap / practice checklist

- Candidates: auto-increment step, UUID, ticket server, Snowflake. Pick Snowflake for 64-bit sortable IDs.
- Layout: 1 + 41 + 5 + 5 + 12 bits. 4,096 IDs/ms/machine, 69 years.
- Watch out for clock sync and unique machine IDs.

## 13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Twitter: Announcing Snowflake
• Flickr: Ticket servers
• Six options for generating distributed unique IDs